GDB Beginner's Guide: The 6 Basics

Debug C/C++ programs with GDB, with every command explained line by line

1. What GDB is for

GDB lets you pause a running program and look inside it. Use it to:

- Find exactly which line causes a crash (segfault)
- Step through code one line at a time
- Inspect variables, pointers, arrays and the call stack
- Stop when a variable changes, or when a condition is true
- Debug a crashed program after the fact using a core dump

You do not need all of GDB. Six commands cover most situations: `break`, `run`, `next`, `print`, `watch`, `bt`. Knowing them means you are never stuck on a terminal-only machine.

2. Setup

On Windows, use WSL2 (Ubuntu) with VS Code's WSL extension. On Linux, `sudo apt install g++ gdb`. On macOS, use LLDB instead (`lldb ./main`); the concepts are identical.

Save as `main.cpp`:

```
#include <iostream>

int sum(int* arr, int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        total += arr[i];
    }
    return total;
}

int main() {
    int data[3] = {10, 20, 30};
    int result = sum(data, 3);
    std::cout << "Result: " << result << "\n";
    return 0;
}
```

Compile with debug info and start GDB:

```
g++ -g -O0 main.cpp -o main
gdb ./main
```

- `g++`: the compiler
- `-g`: embeds line numbers and variable names so GDB can show your source
- `-O0`: no optimization, so code runs in the order you wrote it (with `-O2`, variables show as optimized out)

- `-o main`: names the output file `main`
- `gdb ./main`: loads the program into GDB. It does not run it yet.

You will see a `(gdb)` prompt. Everything below is typed there.

3. The 6 basics

3.1 break: set a breakpoint

```
(gdb) break sum
Breakpoint 1 at 0x1189: file main.cpp, line 4.
```

- `break sum`: pause whenever the program reaches the function `sum`
- `Breakpoint 1`: GDB numbers breakpoints; use the number to delete or disable them
- `0x1189`: memory address of that spot (ignore it)
- `main.cpp, line 4`: where it will pause

Other forms: `break main.cpp:13` or `break 13`. Short form: `b`. Nothing has run yet; you have only told GDB where to stop.

3.2 run: start the program

```
(gdb) run
Breakpoint 1, sum (arr=0x7fffffffe0ac, n=3) at main.cpp:4
4           int total = 0;
```

- `run`: starts the program from the beginning; it runs normally until something stops it
- `Breakpoint 1, sum (arr=..., n=3)`: stopped at breakpoint 1, inside `sum`, with the arguments it received
- `at main.cpp:4`: file and line
- `4 int total = 0;`: the line GDB is paused BEFORE. **This line has not executed yet.**

Short form: `r`.

3.3 next: run one line

```
(gdb) next
5           for (int i = 0; i < n; i++) {
(gdb) next
6               total += arr[i];
```

- `next`: executes the current line, then pauses on the following one
- It steps over function calls: the whole called function runs and you land on the next line. Use `step` to go inside instead.

Short form: `n`. Pressing Enter on an empty line repeats the last command.

3.4 print: inspect a value

```
(gdb) print total
$1 = 0
(gdb) print arr[0]
$2 = 10
(gdb) print total + 5
$3 = 5
(gdb) print arr[0]@3
$4 = {10, 20, 30}
```

- `print total`: shows the current value of `total`
- `$1`, `$2`, ...: GDB stores every result in a history you can reuse, e.g. `print $1 + 1`
- You can print variables, array elements and expressions
- `arr[0]@3`: shows 3 elements starting at `arr[0]`

Short form: `p`. Related: `info locals` shows all local variables, `info args` shows arguments.

3.5 watch: stop when a variable changes

```
(gdb) watch total
Hardware watchpoint 2: total
(gdb) continue
Continuing.

Hardware watchpoint 2: total

Old value = 0
New value = 10
sum (arr=0x7fffffffe0ac, n=3) at main.cpp:5
```

- `watch total`: pause whenever `total` changes, no matter which line changes it
- `continue` (short: `c`): resume until the next stop. `watch` only sets the trap; `continue` lets the program run into it.
- `Old value` / `New value`: shows exactly what changed. Run `continue` again to see 10 to 30, then 30 to 60.

This is the command for "who is corrupting my variable?" bugs, and it is hard to replace with `printf`.

3.6 bt: backtrace (the call stack)

```
(gdb) bt
#0  sum (arr=0x7fffffffe0ac, n=3) at main.cpp:5
#1  0x0000555555555236 in main () at main.cpp:13
```

- `bt`: shows the chain of function calls that led to the current point
- `#0`: the function you are in now (`sum`)
- `#1`: who called it (`main`, at line 13)
- Read bottom to top as the story: `main` called `sum`

When a program crashes, `bt` is usually the first thing to type. To inspect a caller's variables: `frame 1`, then `print result`, then `frame 0` to go back.

4. Finishing a session

```
(gdb) continue     # run until the program ends
(gdb) quit         # leave GDB (short: q)
```

If a watchpoint is still active when its variable goes out of scope, GDB says it is being deleted. That is normal.

5. Cheat sheet

Command	Short	Meaning
`break sum`	`b`	Pause when this function or line is reached
`run`	`r`	Start the program
`next`	`n`	Run one line, stepping over calls
`print x`	`p`	Show a variable or expression
`watch x`		Pause whenever x changes
`bt`		Show the call stack
`continue`	`c`	Resume until the next stop

6. Useful extras

Command	What it does
`step` (`s`)	Run one line, stepping into function calls
`finish`	Run until the current function returns
`info locals` / `info args`	All local variables / arguments
`display x`	Auto-print x every time you stop
`break 6 if i==2`	Conditional breakpoint
`info breakpoints`	List breakpoints and watchpoints
`delete N`	Remove breakpoint number N
`list` (`l`)	Show source around the current line
`up` / `down` / `frame N`	Move between stack frames
`tui enable`	Split-screen source view (Ctrl+X A toggles)

7. Debugging a crash

Simplest way: run the program inside GDB. It stops automatically at the crash.

```
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
(gdb) bt
(gdb) print p
```

For a crash that already happened, use a core dump:

```
ulimit -c unlimited     # allow core files in this shell
./main                  # crashes and writes a core file
gdb ./main core         # load program + core
(gdb) bt
```

On some systems core files are redirected (for example to `coredumpctl`), so the file may not appear in the current folder.

8. Using GDB inside VS Code

The VS Code debugger is GDB underneath, with clicks instead of commands.

1. Install the C/C++ extension
2. Click left of a line number to set a breakpoint
3. Press F5 and choose the C++ (GDB/LLDB) option to generate `launch.json` (make sure it builds with `-g`)
4. Continue: F5. Step over: F10. Step into: F11. Step out: Shift+F11.
5. The Variables, Watch and Call Stack panels correspond to `print`, `watch` and `bt`

The terminal version is worth knowing for servers, remote machines and core dumps.

9. Practice tasks

1. Run this exact session: `break sum`, `run`, `next` three times, `print total`, `watch total`, `continue` three times, `bt`, `quit`.
2. Set `break 14` and check what `print result` shows when you stop there.
3. Change `i < n` to `i <= n`. Use `watch total` and `print arr[i]` to see it read past the array.
4. Add `int* p = nullptr; *p = 5;` at the end of `main`, run it in GDB, and use `bt` to find the crash.

10. Troubleshooting

- **"No symbol table info available":** you forgot `-g`. Recompile.
- **Variables show `<optimized out>`:** compile with `-O0`.
- **Breakpoint never hits:** check the function name or line number, and that you recompiled after editing.
- **Stepping jumps around oddly:** usually optimization is on. Use `-O0`.
- **Next step:** AddressSanitizer catches out-of-bounds bugs automatically, with no stepping.